# FEBAUTO Racing

Lead's car manual

**Part I** Power, networks, a terminal on the car
**Part II** Running a driver, the pad, what to watch
**Part III** When something is wrong, from what actually went wrong

Formula Electric at Berkeley
September 2026, platform release v0.1.0

https://github.com/FEBAutonomous/feb-racing
https://febautonomous.github.io/feb-racing/

# Contents

I Power, networks, a terminal 2

1 Power on, in this order 2

2 Reaching the car 2

II Running a driver 2

3 The command 2

4 The pad 3

5 What to watch, in a second terminal 3

III When something is wrong 3

6 Reflashing 5

Everything here was done on the car between 2026-10-18 and 2026-10-24. Member-facing material is in `member_guide.pdf`; this is the rest.

# Part I
# Power, networks, a terminal

## 1 Power on, in this order

1. Gamepad on. Hold its power button until the LED is solid. *Before* the stack starts: the joystick node looks for the pad once, at start.
2. Battery in, balance plug into the power board, SW1 on. The Jetson boots off the power board; about a minute. If the Jetson is on its USB-C brick instead, SW1 only powers the VESC.
3. A terminal on the car (next section), then the stack (Part II).
4. Off: Ctrl-C the stack, SW1 off, battery out. The Jetson does not need a clean shutdown.

Battery: a 3S LiPo, 12.6 V full, charge below 11.1, never below 10.5. The VESC reports the voltage; the power board runs warm, the battery must not. Store it in the bag, never in the car.

## 2 Reaching the car

The Jetson's login is `feb`, password `feb`. In the order it tries networks at boot:

| Where | How |
|---|---|
| home Wi-Fi `mandem`, or the phone hotspot `pranavphone` | it joins by itself (the hotspot needs the Personal Hotspot page open on the phone until it connects). Then `ssh feb@feb-car` from any laptop on the tailnet, anywhere with internet. |
| nothing known around | 45 s after boot it becomes Wi-Fi network `feb-car`, password `febautonomous`. Join it, `ssh feb@10.42.0.1`. No internet for the car in this mode. |
| USB-C cable to a laptop | `ssh feb@192.168.55.1` |

Gotcha: if the car boots without a known Wi-Fi it goes into its own hotspot and *stays* there; it will not join the phone later. Either power-cycle with the hotspot page open, or join `feb-car` and run `sudo nmcli con up pranavphone`. Adding a new Wi-Fi:

```
sudo nmcli con add type wifi ifname wlP1p1s0 con-name NAME ssid NAME wifi-sec.key-mgmt
    wpa-psk wifi-sec.psk PASSWORD connection.autoconnect yes
```

The car's Wi-Fi region is set to US (5 GHz works). Tailscale name `feb-car`, plain ssh, Tailscale SSH off.

# Part II
# Running a driver

## 3 The command

```
cd ~/FEB/feb-stack && git pull                    # the team's merged drivers
cd ~/FEB/feb-racing && git pull                   # launcher, tracks, docs
```

```
./feb up --car --driver feb_driver --mode-name inspection
```

Drivers come from `feb-stack` first, then the starter kit, so the starter drivers run before anyone has submitted. `--mode-name inspection` caps the supervisor at 0.8 m/s; leave it off for the driver's trackdrive cap. Wait for three lines in the output: *Connected to VESC*, *Opened joystick*, *scan_rotate: lidar_yaw*. Then the pad.

| Driver | What it does on the car |
|---|---|
| `feb_driver` | the gap follower; drove the corridor on 2026-10-19 at 0.8 m/s, door nooks included |
| `feb_racer` | a mapping lap on the gap-follower law at 1.2 m/s, then a raceline at up to 2.5 m/s in trackdrive. Untried on the car as of 2026-10-24: first run in inspection, on a loop it can close, someone walking beside it. |
| `feb_cone_driver` | cones; needs a cone track. |

## 4 The pad

| Hold | |
|---|---|
| LB | MANUAL: left stick forward/back is speed, right stick is steering |
| RB | AUTONOMOUS: the driver drives |
| nothing | IDLE, stopped |
| minus | STOP, latched; LB or RB clears it |

The 8BitDo Ultimate 2C has two modes with different button numbers. The car is configured for the mode it is in (usb id `2dc8:301c`: LB 6, RB 7, minus 10). **Never press the X, B or A + Home combos**; that flips the mode and every button moves. If a bumper does nothing or STOP latches by itself, that is what happened: power the pad off, hold X + Home until it lights, check with `lsusb | grep 8BitDo` that it reads `301c`.

## 5 What to watch, in a second terminal

```
./feb logs -f                                          # everything the stack prints
./feb shell -c "ros2 topic hz /scan"                   # 10 Hz from the lidar
./feb shell -c "ros2 topic echo /feb/state"            # IDLE / MANUAL / AUTONOMOUS /
    STOP and why
./feb shell -c "ros2 topic echo /drive"                # what the driver asks for
```

If `ros2 topic` refuses with *context is invalid* or *failed to initialize wait set*, that is a quirk of the ROS tools after many short commands; the stack is fine. `ros2 daemon stop` and try again, or read `./feb logs`.

Recording a run: `./feb shell -c "ros2 bag record -s mcap -o /home/feb/FEB/bags/NAME /scan /drive /ackermann_cmd /feb/state /odom /joy /sensors/core"` in the second terminal, Ctrl-C to end; the folder lands in `~/FEB/bags` on the Jetson, `scp -r` it to a laptop and open the `.mcap` in Foxglove.

# Part III
# When something is wrong

| Symptom | Cause and fix |
|---|---|
| no *Opened joystick*, or `ls /dev/input/js0` fails | the pad is off, or paired after the stack started. Pad on, LED solid, then Ctrl-C and `feb up` again. The dongle shows `8BitDo IDLE` in `lsusb` while the pad is off. |
| STOP latches on its own; LB or RB does nothing | the pad changed mode (see The pad). |
| no *Connected to VESC*, no `/dev/sensors/vesc` | SW1 off, or the VESC USB in the wrong place. The VESC needs battery power; the Jetson's brick is not enough. |
| `/dev/sensors/rplidar` missing | the lidar's USB; the udev rule maps the CP2102 to it. |
| car drives in the wrong direction / steers wrong | VESC Tool: Invert Motor Direction (the motor), servo 0.3 should steer left. Both set on 2026-10-19. |
| lidar scan is rotated | `lidar_yaw` in `devkit/feb_tools/config/` 1.5708 for the S2 turned 90 degrees. Check: a hand in front of the nose reads 0 degrees in `/scan`. |
| Docker: *Unable to enable DIRECT ACCESS FILTERING, raw table* | the Jetson kernel has no iptables raw module; docker erd runs with |

# 6 Reflashing

Jetson Linux R36.4.3 from this PC with `~/FEB/jetson-flash/prepare_and_flash.sh`: power off, a jumper across J14 pins 9 and 10 while plugging the power in, USB-C to the PC, run the script. The Jetson shows as USB `0955:7523 APX` in recovery. The script pre-seeds the user, hostname, udev rules and the hotspot fallback; Tailscale, Docker's raw-table setting, the xpad driver (compiled from source for the 8BitDo receiver, `~/xpad-src`) and the Wi-Fi networks are done by hand afterwards: `docs/build-car.md`, Session 4, has each command.